



A Pointer-Ownership Model for C Inspired by Rust

David Svoboda
Carnegie Mellon University
Pittsburgh, USA
svoboda@sei.cmu.edu

William Klieber
Carnegie Mellon University
Pittsburgh, USA
weklieber@cert.org

Lori Flynn
Carnegie Mellon University
Pittsburgh, USA
lflynn@cert.org

Ruben Martins
Carnegie Mellon University
Pittsburgh, USA
rubenm@andrew.cmu.edu

Jeffrey Hoskinson
Carnegie Mellon University
Pittsburgh, USA
jphoskinson@cert.org

Abstract

Memory-safety bugs are a major source of vulnerabilities in C code. Much work has focused on spatial memory safety (e.g., buffer overflows), while temporal memory safety (e.g., use-after-free) has received less attention. One solution for achieving temporal memory safety is to apply an ownership model to an existing program and enforce it. In this paper, we describe the design and implementation of a new temporal memory safety model for C source code. Our design improves on CERT's Pointer Ownership Model with enhancements including use of a SAT solver to enforce constraint satisfaction, LLMs to complete a per-program model, and an improved mechanism to prevent use-after-free errors inspired by Rust's borrow checker and object lifetimes. Our implementation performed well on a large test suite of memory-safe and memory-unsafe code examples. We tested all 4,604 C code examples for the 5 CWEs associated with temporal memory safety (CWEs 401, 415, 416, 590, 761) from the Juliet C/C++ test suite. In our tests, all of the memory-unsafe examples were correctly recognized as unsafe, and 81% of the 2,302 memory-safe examples were correctly recognized as memory-safe.

CCS Concepts: • Software and its engineering → Software verification; Automated static analysis.

Keywords: temporal memory safety, C, Rust, static analysis, LLMs, SAT, lightweight formal methods

ACM Reference Format:

David Svoboda, William Klieber, Lori Flynn, Ruben Martins, and Jeffrey Hoskinson. 2026. A Pointer-Ownership Model for C Inspired by Rust. In *Proceedings of the 27th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES '26)*, June 15–16, 2026, Boulder, CO, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3814943.3816182>



This work is licensed under a Creative Commons Attribution 4.0 International License.

LCTES '26, Boulder, CO, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2721-4/2026/06

<https://doi.org/10.1145/3814943.3816182>

1 Introduction

Memory-related bugs such as use-after-free and buffer overflows are among the most prevalent and challenging issues in C and C++ code. Temporal memory safety is particularly important because use-after-free remains highly exploitable in practice and, among memory-safety weaknesses, is second only to buffer overflow in the 2025 CWE Top 25 [34]. These bugs are difficult to detect, often resulting in significant vulnerabilities and system instability. Formal methods, which use rigorous logical reasoning, offer a powerful approach to systematically prove the absence of these types of bugs, providing greater confidence in software reliability and safety. Formal methods are traditionally labor intensive and difficult to scale, with ongoing research making inroads against those challenges [3, 13, 24, 28, 36, 37, 43, 55]. Lightweight formal methods (our work's category) balance rigor with usability, focusing on targeted partial analysis and system verification, often using automated tools and model checking [61].

This paper describes the design of the Pointer Ownership Model (POM), a temporal memory-safety model for C code, together with an approach to automatically check whether a program satisfies this model. The original CERT Pointer Ownership Model (POM) [53] project had similar goals of proving temporal memory safety using responsible and irresponsible pointers. It was a simpler project, with all models built manually. Our design improves on the CERT POM with enhancements that include use of a SAT solver to enforce compliance with the model; a large language model (LLM) to complete a per-program model and models of well-known standard-library functions; a sound mechanism to prevent use-after-free errors, inspired by Rust's borrow checker and object lifetimes; improved function argument handling with a new abstraction of diligent or producer arguments; and handling structs and unions.

The heap is the part of a computer's memory that is used during a program's execution for dynamic memory allocation and deallocation of memory blocks of varying size and varying lifetimes. To avoid heap errors, C and C++ developers must be disciplined in preserving temporal memory safety in the heap. Such discipline requires them to internalize a mental model of how the platform will manage

heap memory as instructed by their code. This model can be informed by literature, such as CERT Recommendation MEM00-C, which specifies to allocate and free memory in the same module, at the same level of abstraction [48]. Technology can also provide constrained models, which we will call “formal models,” such as POM and RAI (see Section 9). We will refer to the contents of files that specify the application of POM to an actual program as a “p-model.” Each p-model is stored in a YAML file. Fig. 1 shows an example C source program, and Fig. 2 shows its associated p-model; details about the program and p-model are available in Sections 2.2 and 2.3.

Our POM design can be used to enforce temporal memory safety. It has the goal of being intuitive for C programmers who work with dynamic memory, since that work requires a mental discipline similar to what our design requires. We developed a conceptual model, a builder, and a verifier for it.

Our design is not intended to verify that every valid C program is temporally memory-safe. Some programs, even well-defined ones, will violate a p-model meant for them. This means that you cannot use our design alone to completely confirm such a program’s temporal memory safety. If a program has pointers that violate its p-model, you must either use an external validation technology or modify the program to comply with its p-model. For example, our design does not handle (1) reference-counted smart pointers, such as C++’s `std::shared_ptr<>`; (2) cyclic pointers, such as in doubly linked lists; (3) Windows handles; or (4) pointers managed by a garbage collector.

However, our design can provide strong memory guarantees: *If all pointers in a program comply with the POM design, then the program is temporally memory-safe.* The POM design is also strong enough to capture a wide range of memory vulnerabilities. For example, using the Common Weakness Enumeration (CWE), our design addresses CWEs 401 [29], 415 [30], 416 [31], 590 [32], and 761 [33].

```

3 // ...
4 void usage(char* msg) {
5     fprintf(stderr, msg);
6     free(msg);
7 }
8
9 int main(int argc, char**
    argv) {
10     char* errormsg;
11     if (argc <= 2) {
12         errormsg = malloc(100);
13         if (errormsg != NULL) {
14             snprintf(errormsg, 100,
15 "Need more than %d
16 arguments!", argc);
17             usage(errormsg);
18             free(errormsg);
19             exit(1);
20         }
21     }
22     // ...
23 }
    
```

Figure 1. Example C source code with a double-free bug

```

Functions:
  usage:
    args:
      msg:
        resp: responsible
        start: [GOOD]
        end: [ZOMBIE]
    return: []
  main:
    args:
      argv:
        resp: diligent
        start: [VALID]
        referent:
          resp: diligent
    locals:
      errormsg:
        resp: responsible
    
```

Figure 2. Example p-model for code in Fig. 1

1.1 Contributions

This paper substantially extends CERT’s earlier Pointer Ownership Model [53] with the following contributions:

1. A formal ownership and borrowing model for C that tracks each pointer’s *responsibility* (ownership), *mutability*, and *state* (*good*, *null*, or *zombie*) at every program point. The model enforces unique ownership of heap blocks, single-writer invariants, and absence of certain memory errors (use-after-free, double-free, invalid free, null-pointer dereference, and memory leaks).
2. A borrowing and lifetime analysis that tracks transitive borrows and enforces the single-writer / multiple-reader property across pointer aliases.
3. A constraint-generation framework that reduces the problem of POM compliance to a boolean satisfiability (SAT) problem. If the generated constraints are satisfiable, the program is temporally memory-safe under the POM design; if unsatisfiable, the POM implementation produces an UNSAT core that localizes the violation. (An UNSAT core is a subset of constraints that together suffice to establish unsatisfiability.)
4. An LLM-assisted *p-model builder* that automatically generates POM annotations (the “p-model”) for each function of a target program. Our tool does not *rely* on the correctness of the p-model produced by the LLM; a bad p-model can result in a false alarm, but never a false assurance of POM compliance.
5. An evaluation on the NIST Juliet C test suite covering five temporal-memory-safety CWEs (401, 415, 416, 590, 761), totaling 4,604 test cases. Of the 4,144 test cases that did not use features not currently supported by POM, our implementation correctly identified 100% of the unsafe examples and 90% of the safe examples. (Our tool recognizes the use of unsupported features and automatically rejects such programs.)

1.2 Data Availability

Our POM tool is publicly available and open-source [52]. Likewise, a self-contained “reproduction artifact” using our tool is also publicly available [51].

2 Overview of POM

2.1 The Model

Our POM design is intended to help developers avoid, identify, and fix temporal memory-safety issues in three stages: 1. Following the mental model should help developers avoid writing code with certain kinds of temporal memory-safety bugs; 2. Automated generation of a p-model reveals some errors for code where the developer forgot to comply with the model; and 3. Our POM implementation’s verifier identifies all remaining p-model compliance errors.

We use the term *heap object* to denote any single chunk of memory that is allocated with the `malloc` or `calloc` or

`aligned_alloc` or `realloc` or `strdup` function. Objects not allocated using one of these functions are not heap objects even if they live on the heap on some platforms. In our design, C pointers are partitioned into responsible pointers, irresponsible pointers, and out-of-scope pointers. Each responsible pointer stewards the object it points to; responsible pointers are intended to point to something on the heap that must be freed. Irresponsible pointers have no involvement with allocating or freeing memory as they may point to any memory (including memory on the stack or into objects), and may also undergo pointer arithmetic. Our design enforces no constraints on out-of-scope pointers; they are outside the scope of the design¹. Their memory safety should be checked by some other mechanism.

If a p-model's state (which indicates if the pointer might be invalid or NULL) disagrees with the use of the pointer in code, then the code violates the p-model and the verifier should detect it. If the user decides that the pointer is out of scope (i.e., managed by some other mechanism), then they should manually update the p-model with this information. The POM implementation's builder and verifier ignore pointers labeled as out of scope.

The terms *responsible*, *irresponsible*, and *out of scope* can be treated like type qualifiers in C (e.g., `const` or `restrict`). They act as qualifiers on pointer variables, irrespective of the variables' values. As with type qualifiers in ISO Standard C, these qualifiers apply to a variable throughout its lifetime. For example, if `p` is considered to be a responsible pointer, it remains responsible throughout its lifetime and cannot cease to be responsible. These terms can apply to local pointers, pointers defined in structs or unions, pointers defined as function arguments, and the return value of a function if it is a pointer type. They can also apply to static pointers (that is, pointers that live in the "global" data segment of memory, regardless of what they point to), but our design does not support static pointers yet.

2.2 Responsible Pointers

Responsible pointers are the subset of pointers that shepherd heap memory and ensure that the memory eventually gets freed. Each chunk of heap memory (i.e., a heap object) may be accessed directly at most by one GOOD responsible pointer. Responsible pointers never point inside the stack, inside the "global" data segment of memory, or inside a heap object except to its beginning.

In ISO C, the value of a pointer can be in one of three states: VALID (i.e., pointing to initialized memory), NULL, or INVALID (i.e., uninitialized or not pointing to good memory) [20]. Like ISO C pointers, responsible pointers have states that can change over their lifetimes. The states for

responsible pointers are GOOD (being the sole good responsible pointer to initialized memory), NULL, and ZOMBIE (uninitialized or freed or pointing to memory managed by another responsible pointer). GOOD pointers in our design are a subset of VALID pointers in ISO C, and ZOMBIE pointers are a superset of INVALID pointers.

It is a violation of the POM design for two GOOD responsible pointers to point to the same heap object. Therefore, assigning one responsible pointer to another and passing a responsible pointer as a function argument are problematic. Both involve copying the address value from one pointer to another. In the POM design, any assignment expression whose right-hand side indicates a responsible pointer can have two interpretations: The left-hand side could indicate an irresponsible pointer; see Section 2.3: Irresponsible Pointers for details. Or the left-hand side could indicate a responsible pointer, in which case the right-hand-side pointer is no longer considered GOOD. While it remains responsible, it has delegated ownership of the pointed-to object to the assigned-to pointer and is now a ZOMBIE. The states of responsible pointers thus employ move semantics [5]. Dereferencing a ZOMBIE pointer violates the POM design, even if the pointer's value remains VALID, according to ISO C.

Likewise, passing a responsible pointer to a function creates the same ambiguity that assignment does. If the function expects an irresponsible pointer as the argument, this is like assigning a responsible pointer to an irresponsible pointer. If the function expects a responsible pointer as the argument, then the function argument receives the same state as the original pointer, and the original pointer becomes a ZOMBIE: It has delegated ownership of the pointed-to object to the function argument, and should not use the pointer, since it has presumably been consumed by the function.

Fig. 1 shows a program with a responsible `errmsg` pointer, as noted by the program's associated p-model in Fig. 2. In `main()`, this pointer begins as a ZOMBIE (uninitialized), becomes VALID before it is passed to `snprintf()`, and then becomes a ZOMBIE again thanks to the call to `usage()`. The program is invalid because it tries to free a ZOMBIE pointer.

2.3 Irresponsible Pointers

Irresponsible pointers are not responsible for cleaning up the memory they point to. Since they do not participate in memory allocation or deallocation, the main concern with irresponsible pointers is that they must respect temporal memory safety. CERT POM had irresponsible pointers but used no tracking mechanism akin to lifetimes, so it did not prevent use-after-free errors. Our POM design does.

An irresponsible pointer cannot be assigned the return value of a function that returns a responsible pointer (such as `malloc()`). Unlike a responsible pointer, an irresponsible pointer can be assigned a value resulting from pointer arithmetic or a value created by C's address-of operator `&`.

¹This is distinct from the ISO C definition of *scope* which applies to blocks of code.

The only liability for an irresponsible pointer is that it might outlast the object it points to, which might cause a use-after-free error. Due to the aliasing problem [45], we do not expect to track the value of every irresponsible pointer. Instead, we track the lifetime of every object that an irresponsible pointer can reference. This is based on Rust’s borrow checker and lifetimes. When we speak of an irresponsible pointer’s lifetime, we refer to the minimum lifetime of whatever object the pointer references. The pointer may be assigned to a different object, but that object must still outlast the minimum lifetime. Rust employs a single-writer / multiple-reader property policy for references, and our lifetime analysis also enforces this policy. This should catch all use-after-free errors, but allows some memory-safe programs to be flagged as non-compliant with POM. (In fact, every object has a lifetime; it is only irresponsible pointers whose objects’ lifetimes must be explicitly tracked.)

By default, irresponsible pointers that are function arguments are assumed to outlast the function call; their lifetimes need not be specified unless their referenced object does not outlast the function call. The same applies to locally declared irresponsible pointers. Irresponsible pointers that serve as function return values are different; they must outlast the function call. Therefore, they must provide an explicit lifetime, which will typically match the lifetime of one of the function arguments. Or they could specify the lifetime “STATIC,” meaning that their object lives until the program terminates.

2.4 Pointer Arguments in Functions

We ignore out-of-scope pointers; they are outside our model.

For convenience, we have two special types of irresponsible pointers for function arguments: diligent and producer.

Irresponsible Arguments. An argument whose value is never freed but might be assigned to point elsewhere should be irresponsible.

Diligent Arguments. A *diligent* argument is an irresponsible pointer that never escapes outside the function, except possibly as the function’s return value. While its contents may be changed, it may not be freed during its lifetime, and neither may the contents of any pointers referenced by it (if, for example, it references an array or struct with pointers). It is therefore loosely analogous to a Rust reference, but with more restrictions. In our experience, most function pointer arguments are diligent. The purpose of diligent pointers is to quickly eliminate analysis of such pointers in functions that treated their data as read-only, such as `strlen()`.

Producer Arguments. Another type of irresponsible pointer is a *producer argument*. It is a pointer-to-pointer argument where the outer pointer (i.e., the producer argument itself) is irresponsible, but should not be changed inside the function. However, the inner pointer (i.e., the pointer obtained

by dereferencing the producer argument) is responsible, and might be allocated, freed or otherwise modified. It is therefore loosely analogous to a Rust mutable reference. Like Rust mutable references, a producer pointer must be the only pointer through which its pointed-to data may be changed. Producer arguments are intended to capture an “output parameter” pattern where the address of a pointer is passed in and the callee writes a responsible pointer value in the provided memory location. One example of a function with a producer argument is the GNU `getline()` function.

Responsible Arguments. A responsible function argument is the only pointer in scope to the data in question. A function that takes a responsible pointer that starts as GOOD but becomes a ZOMBIE is said to “consume” the argument. A responsible pointer might also be consumed by assigning its value to another responsible pointer. Generally, a function argument pointer should be declared “responsible” only if the function consumes the argument.

A function with one or more responsible pointer arguments may change the arguments’ states differently based on whether or not the function produced an error. For example, the `realloc()` function takes a responsible pointer as its first argument. If it was GOOD and `realloc()` returns a non-NULL pointer, it becomes a ZOMBIE. If `realloc()` returns NULL, the pointer’s state remains unchanged. POM does not capture this sort of behavior in general. However, since `realloc()` is commonly used, we have implemented a special case to handle it. See Section 8 for more information about handling `realloc()` in a more extensible fashion.

2.5 Control Flow and Responsible Pointer States

When considered across multiple executions of a function, a pointer can be in multiple states. For example, `free()` can handle pointers that are GOOD or NULL. POM assumes that the set of states a pointer is in can be determined statically. For any two states, branching can create a pointer that could be in both states. For example, `malloc()` returns a responsible pointer that could be GOOD or NULL.

This can be a source of trouble. In ISO C there is no way to distinguish GOOD responsible pointers from uninitialized pointers. This (among other things) requires a developer to maintain internal discipline to make sure that only valid pointers are passed to most library functions. The POM tool keeps track of the states of pointers and issues warnings. For example, our verifier will warn if a pointer that might be uninitialized or NULL is dereferenced.

3 Properties of the Model

Now, let us formalize what was discussed more informally in the previous section.

3.1 C-Paths and Locations

The POM design uses *C-paths* to identify/name pointers in the program. A *C-path* is rooted at a variable, function argument, or return value, and may include dereference steps and struct/union field projections. Concretely, we use the following forms for C-paths:

- *func::locals::var_name*
- *func::args::arg_name*
- *func::return*
- **cpath* (dereference of *cpath*)
- *cpath.field_name* (address of field in struct)

Note we write “*cpath.field*” to denote the *address* of the field rather than the value stored in the field. (In contrast, in C, “*p.field*” is syntactically invalid if *p* is of pointer type; instead, the syntax “*&(*p).field*” is used for the address of a field.)

A heap object that cannot be referenced by any C-path indicates a memory leak. If no memory leaks exist in a program at a point in time, then every heap object has at least one C-path to reference it.

In the implementation of our verifier, local variables are not variables in the C source code but rather LLVM virtual registers (identifiers that appear on the left-hand side of an LLVM IR [1] instruction “*var = opcode ...*”). Since the *mem2reg* LLVM pass for converting to static single assignment (SSA) form can sometimes make it hard to map an LLVM virtual register back to a variable in the source code, we insert a custom pass before the *mem2reg* pass to preserve information and suppress certain optimizations that *mem2reg* would otherwise perform.

We use the term “location” to refer to a memory location (heap object), an LLVM virtual register, the current function’s return value (“*func::return*”), or an argument to the current function (“*func::args::arg_name*”). Note that multiple C-paths may refer to the same memory location.

We say that two C-paths are *pointer aliases* of each other iff they point to the same memory location. We define the term *location alias* as follows:

- For a C-path **p*, the location aliases of **p* are all C-paths **q* where *q* points to the same memory as *p*.
- Any other type of location (an LLVM virtual register, a function arg, etc.) has only itself as a location alias.

3.2 Type-Qualifier Properties

The following properties act like type qualifiers on a C-path; they do not vary by program point.

responsible(*p*): The C-path *p* *owns* the memory it points to and is responsible for freeing it or transferring ownership to another responsible C-path.

mut(*p*): The C-path *p* may be used for writing to memory (i.e., “**p = ...*” is permitted).

out_of_scope(*p*): The C-path *p* is excluded from the POM design’s analysis (e.g., for reference-counted pointers). This is not yet implemented in our tool.

Earlier, the terms “diligent” and “producer” were discussed. For constraint-generation and verification, “diligent” and “producer” are not first-class concepts; we treat diligent pointers as immutable irresponsible pointers, and we treat producer arguments as mutable irresponsible pointers.

3.3 State Properties

A *program point* is a point immediately before or immediately after a given statement (LLVM IR instruction) is executed. Given a statement *S*, we write S^{pre} to denote that program point before *S* and we write S^{post} to denote the program point after *S*.

The POM design has the following properties, for C-path *p* and program point *t*:

good(*p, t*): *p* may hold a valid (initialized, non-null, non-freed) address at program point *t*. (The purpose of this property is for ensuring the absence of memory leaks; at the end of a responsible C-path’s lifetime, we check that it is not good.)

null(*p, t*): *p* may be NULL at program point *t*.

zombie(*p, t*): *p* may be a zombie at program point *t*. A C-path is a zombie if the pointer is uninitialized or it is freed or its value has been assigned to another responsible pointer variable.

These state properties are over-approximations (i.e., a “may” analysis rather than a “must” analysis): if the property is false, then the corresponding state *definitely* does not hold on any execution.

Copying a responsible pointer to another responsible C-path *moves* ownership: the source becomes a zombie and the destination becomes the new owner. Copying to an irresponsible destination does not zombify the source. The return C-path of a function (e.g., *foo::return*) is a valid responsible destination, allowing functions to return ownership.

3.4 Key Invariants

The POM design enforces the following invariants:

1. **Unique owner.** For every live heap block *B* and at every program point, exactly one location holds a good, responsible pointer to that block.
2. **Single writer.** For every memory location and at every program point, at most one C-path can be used to write to it (or free it); that C-path must be mutable and have no outstanding borrows.

4 Borrowing and Lifetimes

The POM design includes a borrowing analysis inspired by Rust’s borrow checker and non-lexical lifetimes (NLL).

4.1 Borrow Tracking

When a pointer value is copied from a source C-path to a destination C-path, POM records a *borrow* from source to destination. Borrows are tracked transitively: if p_2 borrows from p_1 and p_3 borrows from p_2 , then p_3 transitively borrows from p_1 . For pointer-to-pointer types, borrows propagate to deeper dereference levels (e.g., if p_1 and p_2 are pointers to pointers, and p_2 borrows from p_1 , then $*p_2$ borrows from $*p_1$). The borrow information is used for alias analysis; when a mutable C-path is used to write to memory, we can identify its potential aliases from the borrow information. We track both *may-borrow* and *must-borrow* information, to enable doing both *may-alias* and *must-alias* analysis.

4.2 Borrow Constraints

- A C-path may not be *read* through if it (or any C-path rooted at it) has any outstanding *mutable* borrows.
- A C-path may not be *written* through or *freed* when it (or any C-path rooted at it) has any outstanding borrows (mutable or immutable).
- When an instruction makes a direct borrow from a C-path *src* to a C-path *dest*, if there is already a direct borrow to another C-path *other_dest* from any C-path rooted at the root of *src*, then both *dest* and *other_dest* are constrained to be non-mutable. However, reborrowing thru a chain is permitted, analogously to Rust.

Multiple direct borrows	Borrow chain
$p_2 = p_1;$	$p_2 = p_1;$
$p_3 = p_1;$	$p_3 = p_2;$
$*p_3 = 42; /* \text{violation} */$	$*p_3 = 42; /* \text{allowed} */$

4.3 Lifetime Analysis

Borrow lifetimes are determined by a classic dataflow liveness analysis on the LLVM IR, similar to non-lexical lifetimes in Rust. When a borrowing variable becomes dead, its borrows are removed, potentially unblocking writes or frees through the source C-path.

4.4 Interprocedural Analysis for Borrowing

For each function, a function summary is created to indicate the borrowing relations introduced by the function. This summary is then used at call sites of the function. If this summary is created by the LLM-based builder (or by another tool, or manually), then the verifier will verify it. If operating without a provided summary, then the LLVM analysis tool will do a whole-program fixed-point analysis to create the summary, to handle cases of mutually recursive functions.

5 Constraint Generation

Our verifier generates a set of constraints that, if satisfied, imply that the program meets certain temporal memory-safety properties (in particular, absence of use-after-free errors, double-free errors, and memory leaks). The constraints are

encoded as a boolean satisfiability (SAT) problem. Properties such as “*responsible(foo : args : x)*” are represented as boolean variables in the SAT problem.

Notation: We write “ $x \rightarrow y$ ” to denote material conditional (i.e., logically equivalent to “ $\neg x \vee y$ ”).

5.1 Responsible implies Mutable

Every responsible C-path is constrained to be mutable.

5.2 Control-Flow Constraints

For two IR instructions P (predecessor of S) and S (successor of P), and for each live C-path ptr , we propagate state:

$$\text{good}(ptr, P^{\text{post}}) \rightarrow \text{good}(ptr, S^{\text{pre}})$$

$$\text{null}(ptr, P^{\text{post}}) \rightarrow \text{null}(ptr, S^{\text{pre}})$$

$$\text{zombie}(ptr, P^{\text{post}}) \rightarrow \text{zombie}(ptr, S^{\text{pre}})$$

with exceptions for null-check branches and `realloc`.

5.3 Instruction-Specific Constraints

We generate constraints for each LLVM IR instruction class.

Stack allocation ($p = \text{alloca}(\dots)$). The result p is an irresponsible pointer that is good. If p is a pointer-to-pointer, then $*p$ may be zombie or null.

Heap allocation (`malloc`, `calloc`). The result is a responsible pointer that may be good or null but is not a zombie. For a pointer-to-pointer variable p :

- For “ $p = \text{malloc}(\dots);$ ”, $*p$ may be zombie or null.
- For “ $p = \text{calloc}(\dots);$ ”, $*p$ is null.

Free. The argument to `free()` must be responsible and not a zombie. (It may be null.) After the call, this pointer becomes a zombie if it was good. If the memory being freed holds a responsible pointer, then the held pointer must not be good (to prevent a memory leak).

Pointer-Copying Constraints. Many LLVM IR instructions (loads, stores, phi nodes, function calls, returns) copy a pointer value from a source C-path to a destination C-path. Our verifier generates a similar set of pointer-copying constraints for each such operation.

The constraints fall into four groups:

1. **Responsibility and Mutability**. If the destination is responsible, then the source must also be responsible. If the destination is mutable, then so must the source.
2. **Move semantics at the source**. If the destination C-path is responsible, then copying a good pointer value out of the source must zombify the source, enforcing unique ownership. For pointer aliases of the source C-path that are only *may* aliases (as opposed to *must* aliases), the alias’s good status is conservatively preserved (in addition to gaining zombie status if the destination C-path is responsible). For the source C-path itself and *must* aliases, the good status is dropped

if the destination C-path is responsible. The source C-path must not have any outstanding borrows if the destination C-path is responsible.

3. **State propagation to the destination.** The destination and its location must-aliases inherit each of the source’s state properties (good, null, zombie). A location may-alias ORs its prior value with the source’s.
4. **Deep-copy constraints.** For pointer-to-pointer types, the constraints propagate recursively through dereference levels. If *src* and *dest* are of type ***T*, then copying *src* to *dest* also connects **src* to **dest*: their responsibility qualifiers must agree ($\text{responsible}(*src) \leftrightarrow \text{responsible}(*dest)$), mutability must be compatible ($\text{mut}(*dest) \rightarrow \text{mut}(*src)$), and the state properties of **src* propagate to **dest* and its aliases. This continues to deeper levels, up to the pointer depth.

Load/Store. The pointer used for reading or writing to memory must not be null or a zombie. Writing to memory additionally requires the pointer to be mutable. Reading or writing a pointer value generates pointer-copying constraints.

If a non-mutable pointer is used for reading a value of pointer type from memory, then the result is non-mutable, even if the read memory location holds a mutable pointer. So, e.g., for a statement such as “***p* = 5;”, both *p* and **p* are constrained to be mutable.

Function Calls. Arguments are connected to formal parameters via pointer-copying constraints. Memory writes inside the callee are captured by a function summary so that they may be reflected at the callsite. If the function returns a value, then the return value at the call site is connected via pointer-copying constraints to the value of the *ret* instruction in the callee. (The LLVM IR is normalized so that each function has exactly one *ret* instruction.)

Although not strictly necessary for soundness, our verifier enforces a constraint that zombie values may not be passed as arguments. This enables better localization of constraint violations and enables better partial handling of system library functions for which models are not provided.

Dead Variables. When a responsible variable *x* dies, the POM design requires $\neg \text{good}(x)$ at that point, to prevent memory leaks.

5.4 SAT Solving and Diagnostics

The generated constraints are converted to DIMACS CNF format and solved using CryptoMiniSat [49]. If satisfiable, the satisfying assignment constitutes a valid p-model and the program complies with it. If unsatisfiable, we extract an UNSAT core from a DRAT proof [57]. We also have a script that prompts an LLM to explain the constraint violation in more human-understandable terms. The prompt for the LLM includes the UNSAT core, the LLVM IR representation, the

program source code, and a description of POM. In limited ad-hoc testing, we have found that this works for small programs; additional engineering work would be needed to handle large codebases.

Soundness. See Appendix A for a limited proof sketch that the constraints described in this section (together with those in 4.2) suffice for detecting use-after-free and double-free errors. We outline a proof for a toy LLVM-like instruction set; for simplicity, we omit *realloc* and special handling of null-check branches. The proof sketch covers use-after-free and double-free but not memory leaks.

6 Implementation

Fig. 3 shows the POM implementation architecture. The implementation stores each program-specific model (p-model) in a YAML file and supports two automated construction paths in addition to manual modeling. The first path is an LLM-based builder. The second path is a SAT-builder method that reconstructs a p-model from a satisfying assignment produced by the verifier.

In practice, for a multi-file codebase with a makefile, the workflow begins by using Bear [39] to produce a compilation database, then invoking Clang [1] to generate LLVM IR.

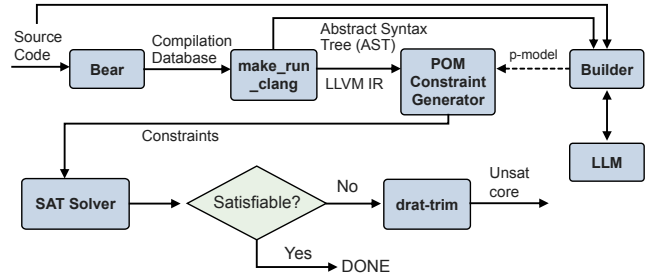


Figure 3. POM Implementation Architecture

The implementation supports both single-function checking and whole-program analysis. In whole-program mode, multiple LLVM IR files are linked before constraint generation. External and standard-library calls can be handled by supplying p-models for those functions.

The LLM builder is prompted with the source code for each function and must return information about each pointer in that function in a JSON format. These JSON snippets are used to complete the p-model, which should include any library functions used by the code. The LLM builder can be used to create p-models for well-known standard library functions; in our experience (with Standard C and POSIX libraries), it is highly accurate.

We hypothesize that an LLM may be better at discerning programmer intent than static analysis alone, especially if the code is defective or fails to comply with its p-model. A risk of using LLMs is that they sometimes hallucinate.

However, we do not *rely* on the LLM, since the verifier will assess the accuracy of a p-model and will indicate an error on any p-model that the program does not comply with. A hallucinatory LLM is still useful in that it may generate correct p-models occasionally, thereby saving developers from creating them manually.

Note that a “trivial” p-model could be constructed that declares all pointers as out-of-scope. This complies with POM, but proves nothing about temporal memory safety. When building the p-model, the LLM is forbidden from declaring pointers as out-of-scope; it must assign some other responsibility to each pointer.

7 Evaluation

To support reproducibility, we have made the implementation publicly available [52], along with a “reproduction artifact” using our implementation [51].

The Juliet C/C++ test suite [6] is organized by CWE. Its test cases are self-identified as GOOD or BAD, where the BAD examples violate the CWE in question, and the good examples do not. Using the model o4-mini-2025-04-16 [42] as the LLM for building the p-model on the C tests (not the C++ tests), we evaluated the implementation on the five Juliet CWEs associated with temporal memory safety: 401 [29], 415 [30], 416 [31], 590 [32], and 761 [33]. This totals 4,604 examples (2,302 GOOD and 2,302 BAD).

7.1 Supported Test Cases

Our implementation does not yet support all C features. In particular for Juliet, we do not support writes to global variables of pointer type, writes to struct/union fields of pointer type (when the struct/union is accessed from multiple functions), and certain pointer casts (e.g., casting `int**` to `int*` and then casting back to `int**`). Some of the Juliet test cases use function pointers, but our implemented analysis is sufficient to handle them. Out of the 4,604 total test cases, our implementation supports 4,144 (2,072 GOOD and 2,072 BAD). Unsupported test cases are conservatively treated as non-compliant with their p-models.

We did two evaluations with Juliet: one where we used the LLM-based builder to construct p-models and the SAT-based verifier to confirm if they are satisfiable, and another where we used the SAT-based analyzer to both build the p-models and verify them, without using an LLM.

Figure 4 breaks the Juliet C results down by CWE and by GOOD/BAD outcome counts, for tests with the LLM builder then SAT verifier. In that figure, *Successful GOOD* denotes a GOOD example for which the tool successfully generated or validated a compliant model, while *Successful BAD* denotes a BAD example for which the tool correctly concluded that no compliant model exists. The counts show successful handling of all BAD cases across all five CWEs.

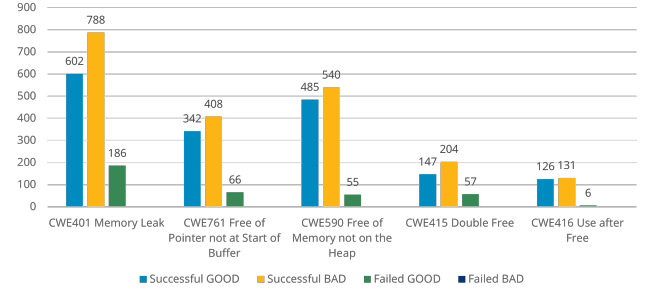


Figure 4. LLM builder, then SAT verifier: Juliet test-case outcomes for the five temporal-memory-safety CWEs. Successful GOOD and Successful BAD denote cases for which the tool correctly accepts GOOD examples or rejects BAD examples, respectively.

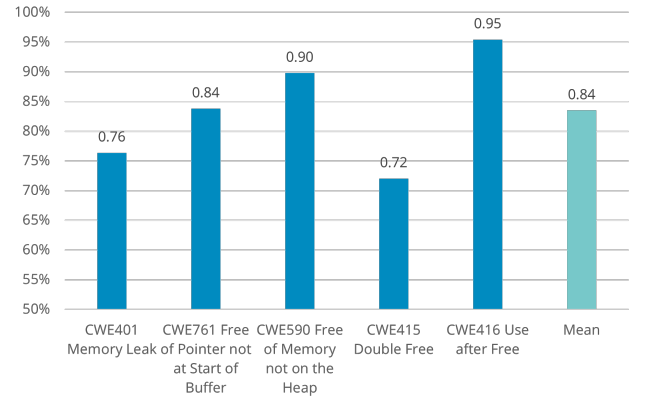
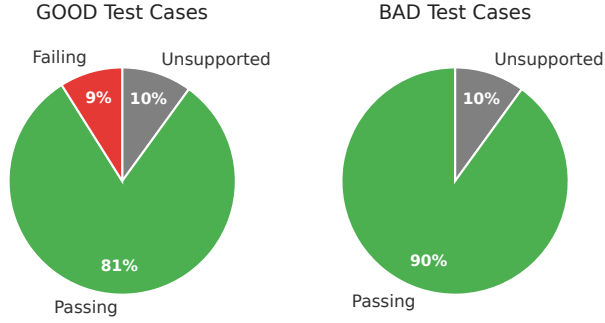


Figure 5. Percentage of supported GOOD Juliet C test cases where the LLM builder provided a valid p-model.

Figure 5 reports the percentage of supported GOOD examples that were correctly recognized by the POM implementation as memory-safe, using the LLM-based builder to provide the p-model. This is highest for CWE-416 at 95% and lowest for CWE-415 at 72%, with a mean of 84%.

Figure 6 shows that our whole-program SAT-based analyzer supported 90% of the temporal memory safety Juliet C tests for both the GOOD and BAD sets of test cases. Our implementation correctly recognized every BAD example as memory-unsafe, and recognized 81% of all GOOD examples (90% of supported GOOD examples) as memory-safe. (Unsupported examples are conservatively treated as memory-unsafe.) Test failures (9% in Fig. 6a) can result if the program fails to comply with its p-model but is still memory-safe or from imprecision of static analysis.

The evaluation also included a manual comparison study on the `dos2unix` codebase (version 7.5.2) to assess automated p-model construction on a real program. The `dos2unix` code builds on Windows or Linux (with `#ifdefs` for both). It has 4 kLoC, 49 functions, and 169 pointers (local, function



(a) GOOD test cases (no vul). (b) BAD test cases (vul present).

Figure 6. Whole-Program SAT Solver: Juliet C test suite results. “Passing” means the tool correctly identifies the presence or absence of the CWE.

arguments, or return values). There were no arrays of pointers², structs, unions, or static pointers. Our study focuses on the Linux build. We manually constructed a p-model for the codebase, which serves as a reference model. We then applied the LLM-based builder to the same codebase and compared the generated p-model against the reference model. In our study, the LLM correctly labeled 159 of 169 pointers (94.1%). Table 1 details the ten LLM labeling errors.

These experiments support the broader claim that LLM-assisted p-model completion can substantially reduce modeling effort while SAT-based verification preserves trustworthiness by rejecting inconsistent models and exposing an UNSAT core when the model and program do not agree.

7.2 Reasons for Failures

We used gpt-5.4-mini-2026-03-17 (with reasoning=high) to analyze the failing GOOD test cases in Juliet. (Yes, this is a different model than the one we used to generate the p-models.) The LLM was provided with the source code, the LLVM IR, the UNSAT core, and a description of POM. We manually audited a subset of the results and confirmed that the LLM was providing correct analysis. Table 2 details the main causes of POM failures on GOOD test cases; these cover 166 of the 206 failures on GOOD test cases. 95 of these 166 failures represented code that was actually memory-safe, but violated POM constraints, and the remaining 71 failures used code that foiled our static analysis.

8 Limitations

The POM design currently has the following limitations:

- **Indirect calls (i.e., calls via function pointers).** We do a simple interprocedural analysis to try to discover the possible target functions. The indirect call is then replaced by a nondeterministic switch (with one branch

for each possible callee). Our analysis flags cases that it cannot handle, so that a callsite can be marked as possibly missing potential callees. If no possible targets are found, the call is treated as a no-op.

- **Interprocedural structs.** Our implementation currently does not support structs with pointer fields that are accessed across multiple functions.
- **Recursive data structures.** Linked lists, trees, and other recursive heap structures are not yet supported. We have designs and test examples for singly linked lists and supporting them is future work. Doubly linked lists are not currently supported by our design.
- **Independent field borrowing.** Borrowing a field of a struct currently borrows the entire struct; Rust-style per-field borrowing is future work.
- **Generalizing realloc().** Rather than having code to handle realloc() specifically, generalize the POM p-models to specify different end states for function success and failure.
- **Interior scopes.** Pointers to locals that escape an inner scope but not the function are not caught, because LLVM IR does not preserve inner-scope boundaries.
- **Unions.** Pointers in unions require further modeling.
- **Globals.** We currently do not support static pointers.
- **Multi-threading.** So far, we have considered only single-threaded code.

9 Related Work

Memory-safety violations are a major source of vulnerabilities and have been extensively studied [10–12, 54]. These primarily include temporal and spatial errors, as well as invalid pointer values, memory leaks, and segment confusion. In this work, we focus on temporal memory safety by tracking pointer ownership and checking compliance with temporal safety constraints.

Several prior formal models have addressed temporal memory safety, with our approach specifically inspired by three main directions [2, 50, 53]. First, our approach builds on CERT POM [53], which allows developers to formally specify temporal memory-safety constraints via a p-model, which can be partially inferred using static analysis and automatically validated. We extend POM by enhancing its memory-safety coverage through explicitly tracking pointer lifetimes.

Second, our approach is influenced by Rust’s borrow checker, which ensures comprehensive memory safety, including thread safety and controlled concurrency [2, 22]. Our approach supports a limited set of Rust features, specifically those similar to Rust’s Box pointers and references, but does not cover Rust’s reference-counted pointers. Translation-based approaches such as DARPA TRACTOR [15] convert C to Rust, but may use unsafe Rust code, thereby undermining Rust’s safety guarantees. Crown infers ownership constraints from C2Rust [19] output and solves them with SAT

²For the Windows build, there are copies of the argv array, but they are #ifdefed out in the Linux build.

Table 1. LLM Builder on dos2unix: The 10 LLM Labeling Errors for dos2unix 7.5.2

Error Count	Type	Variable	Notes
1	State	fname_ret	The state is NULL vs. NULL ZOMBIE. (But fname_ret is involved in a p-model violation.)
2	Responsible	rFN, rVal	The file is assigned from realpath(), which violates the p-model.
2	State	lnF, f	The LLM does not know lnF/f should be passed to fclose().
1	State	argv	The LLM thinks it can be NULL but is passed to ungetc(), which requires a VALID pointer.
1	Responsible	main(argv)	The main(argv) variable should always be diligent.
3	Responsible	main	The LLM is confused by #ifdefs. Future work: Fix using undef or preprocessed source.

Table 2. Error Classification for Juliet

Error Count	Summary	Notes	Type
55	Borrow/write conflict	Here, &data is taken twice (dataPtr1 = &data; dataPtr2 = &data;) and then data is written to directly while the borrows are still live.	Memory-safe, but violates the p-model
40	Returning a pointer to stack/local storage	A function returns an invalid pointer, which pointed to stack space provided by alloca(), but that pointer is never used.	Memory-safe, but violates the p-model
38	Loop Imprecision	Has a loop that iterates exactly once (as in for(i = 0; i < 1; i++)). Memory safety depends on the number of iterations being exactly one.	Insufficient static analysis
33	free() guarded by global or static flag	The call to free() is guarded by a conditional such as “if (global_or_static_flag)”, so the flag is always true.	Insufficient static analysis

to refactor raw pointers into safer Rust pointer types [62]. Its monotonicity assumption avoids alias analysis and enables high scalability. In contrast to translation approaches, POM can be used when the code must remain C.

Third, our approach aligns conceptually with the Resource Acquisition Is Initialization (RAII) paradigm from C++ [7, 14, 50]. RAII systematically associates resource lifetimes—such as dynamically allocated memory, file handles, and mutex locks—with object lifetimes, encapsulating resource acquisition in constructors and resource release in destructors.

Other approaches to guarantee memory safety in C include extending the language with memory-safe pointers [38, 63, 64]. For instance, Checked C [16] extends C by introducing checked pointers that explicitly specify bounds for objects and arrays, allowing compile-time verification or runtime checks. Even though some semi-automated approaches have been suggested to transpile C into Checked C code [26], this method still requires rewriting large parts of the code base into a safe language, which limits its practicality. The MSA tool helps port C code to Checked C [35], and like our builder and verifier, it uses compiler-generated ASTs,

static analysis, and LLMs to deal with complexity. MSA also uses symbolic inference, unlike our system. Zhou et al. extend Checked C with temporally safe fat pointers that use in-place keys and object locks, checking key-lock agreement dynamically on dereference [64]. Their design improves fat-pointer practicality by exposing distinct safe and legacy pointer types for incremental compatibility; POM instead leaves ordinary C pointer representations unchanged and checks ownership/lifetime constraints statically, imposing no runtime metadata or checks.

Silva et al. pursue a “Rust borrow checker for C” via source-to-source transpilation, annotations, and MIR-like normalization, using an intraprocedural NLL-style analysis to enforce Rust-like ownership and aliasing constraints [46]. Instead of transpilation, POM records a C-specific ownership/lifetime discipline in a separate p-model and uses SAT to check the original C program for temporal-safety violations.

Alternatively, C-Rusted [4] incorporates Rust-inspired concepts like ownership and borrowing through standard C macros that vanish after preprocessing, enabling static analysis without altering compiler behavior. Its analyzer detects

errors such as NULL pointer dereferences, memory leaks, and use-after-free, but extensive manual annotations limit scalability for large legacy codebases. Similarly, SafeMD applies Rust-inspired ownership to C for repair: assuming the input already satisfies exclusive ownership, it tracks each allocation's owner, inserts free at last-use candidates, and selects patches via exact cover [59]. POM is complementary and broader: it does not repair leaks, but checks an explicit p-model for leaks, double frees, invalid frees, null dereferences, and use-after-free without first translating or rewriting the program into an exclusive-ownership subset.

Cyclone [21] and CCured [40] are influential approaches to memory safety in C. Cyclone introduces a safe dialect with region-based memory management and unique/borrowed pointers, while CCured retrofits legacy code using whole-program inference and hybrid static/dynamic checking.

Another approach is to extend pointer structures to include bounds information, inserting run-time checks during compilation to ensure memory accesses remain within allocated regions [23, 47]. While effective, this modification introduces significant run-time and memory overhead [23].

Other approaches to temporal memory safety build on the low-overhead Capability Hardware Enhanced RISC Instructions (CHERI) architecture [17, 18, 27, 56, 58], which provides spatial memory safety and has ongoing standardization efforts with significant industry involvement [27, 56]. CHERI enforces memory safety using capabilities with hardware support. These approaches are not available on mainstream CPUs (e.g., Intel x86-64 and AMD64) since CHERI requires capability registers, tagged memory, or hardware-enforced bounds checks for pointers. CHERI dynamically traps during execution to prevent memory safety violations, while the POM implementation detects potential temporal memory safety violations in the code so code can be fixed prior to execution - both POM and CHERI could be used.

Model checking techniques [8, 9, 25, 41, 44, 60] represent a general approach for verifying memory properties but face scalability challenges due to their precise modeling of program memory. In contrast, our approach, although targeting only a subset of memory-safety properties, aims to be lightweight, scalable, and efficient.

VIP targets a different design point: it verifies C programs with integer-pointer casts by using a verification-oriented memory model in which programmers make pointer provenance explicit, e.g., via `copy_alloc_id` [25]. POM instead targets temporal heap-safety properties: it records ownership/lifetime intent in a p-model and uses SAT to check the C program, rather than modeling low-level provenance through integer-pointer casts.

10 Conclusion and Future Work

This paper describes our approach to demonstrate that our POM design can be used to prove partial temporal memory

safety in C code. We highlighted and detailed our POM design, code evaluation, tool design methodology, validation tool, and testing. We developed code for automating p-model creation (via SAT solver and via LLM) and validation (via SAT solver) as well as an automated testing framework to run experiments. We tested how accurately the SAT solver and LLM populated the p-model for some codebases.

Using an LLM for p-model generation support is intended to lower manual effort and increase correctness, but it risks hallucinations that cause the verifier to say that the program does not comply with the p-model. Our test results provide indications that despite some hallucinations, automation of p-model generation and validation is feasible with limited manual effort. Our design is intended to help developers avoid, identify, and fix temporal memory-safety issues with its mental model, automated p-model generation, and automated verification. If successful, our design could improve the security and functionality of much of the huge amount of C code currently in use, at a low cost (due to automation) and with no performance reduction.

Future work will investigate the effectiveness of different LLMs and prompt designs for p-model generation. We plan to study interactions between out-of-scope pointers and responsible/irresponsible pointers in data structures such as doubly-linked lists and reference-counted pointers, and to extend our design to support these cases. Finally, we aim to expand C coverage by supporting static/global pointers and pointer tracking through integer and non-pointer casts.

Acknowledgments

We wish to thank Robert Seacord, Martin Uecker, Aaron Ballman, and Dave Banham, as well as our anonymous reviewers, for their feedback that helped improve this paper.

Copyright 2026 ACM. This material is based upon work supported by the Department of War under Air Force Contract No. FA8702-15-D-0002 with Carnegie Mellon University for the operation of the Software Engineering Institute, a federally funded research and development center.

References herein to any specific entity, product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by Carnegie Mellon University or its Software Engineering Institute nor of Carnegie Mellon University - Software Engineering Institute by any such named or represented entity.

[DISTRIBUTION STATEMENT A] This material has been approved for public release and unlimited distribution. Please see Copyright notice for non-US Government use and distribution.

CERT® is registered in the U.S. Patent and Trademark Office by Carnegie Mellon University.

DM26-0506

A Proof Sketch of Soundness

We want to prove: If the program satisfies all POM constraints (see Sections 5 and 4.2) with no out-of-scope pointers, then no use-after-free or double-free errors occur.

For simplicity, we will work with the following small LLVM-like IR instruction set (no structs, unions, or arrays):

- `t = malloc()`
- `t = load p`
- `t = call(a_1, ..., a_n)`
- `br label`
- `br nondet(), label_if_true, label_if_false`
- `t = phi [v_1, label_1], [v_2, label_2], ..., [v_n, label_n]`
- `free(t)`
- `store t, p`
- `t_2 = t_1`
- `return t`

We make the following simplifying assumptions:

- `malloc()` always allocates a fresh address (never NULL or an address of previously allocated memory). Each allocated heap block is exactly big enough to hold a single pointer.
- All called functions, other than `malloc()` and `free()`, are defined by the program. (So, e.g., no `realloc()`.)
- The liveness analysis (for determining lifetimes of variables) is correct.
- Function summaries are correct; that is, each summary safely over-approximates the state changes and borrow relations induced by the corresponding function.
- Each IR variable has an associated type that records pointer depth (e.g., distinguishes `int*` from `int**`), and the program is well typed.
- Every IR variable is defined before being used, so there are no uses of uninitialized variables. (However, the contents of `malloc'd` memory is uninitialized.)
- The entry point `main` takes no arguments.

Concrete states. A concrete execution state consists of (1) a mapping of locations to values and (2) a mapping of allocated blocks to either live or freed. A pointer value is either NULL, uninitialized, or the address of a heap block.

Live C-paths. We say that a C-path p is *live* in a concrete state if (1) the root of p is live and (2) every dereference needed to evaluate p follows a pointer to a live block.

Effective Mutability. Suppose that a pointer-valued variable p_1 points to another pointer p_2 in memory. Recall that, when loading p_2 into a variable v via “ $v = \text{load } p_1$ ”, if p_1 is non-mut, then v is constrained to also be non-mut even if p_2 is mut (mutable). So, even if the C-path $*p_1$ is marked mut, it is *effectively* non-mutable if p_1 is non-mut. In light of this, we define a notion of *effective mutability* ($emut$) as follows:

$$emut(x) = \begin{cases} mut(x) & \text{if } x \text{ is root of C-path} \\ emut(p) \wedge mut(*p) & \text{if } x = *p \end{cases}$$

We say that a C-path is *write-enabled* if it is effectively mutable, is not marked as a zombie, and has no outstanding borrows.

Notation for dereference chain. Let us write “ $*^i p$ ” to denote a C-path rooted at p with i dereferences. So, $*^0 p$ is p , $*^1 p$ is $*p$, $*^2 p$ is $**p$, etc.

Use-after-free and double-free events. A use-after-free error can occur only when an instruction dereferences a pointer whose concrete value is the address of a freed heap block. The dereferencing instructions are load and store. A double-free error consists of calling `free(t)` on an already-freed block. Thus, it suffices to prove:

1. immediately before every load p and store t, p , the pointer p does not point to a freed heap block; and
2. immediately before every `free(t)`, the pointer t does not point to a freed heap block.

A key observation is that the generated constraints require dereferenced pointers (and pointers passed to `free`) to not be zombies. Therefore, if we prove that every live C-path whose concrete value is a dangling address is abstractly marked zombie, then we have established that use-after-free and double-free are impossible.

We proceed by induction on the execution trace. Let the trace be

$$\tau_0 \rightarrow \tau_1 \rightarrow \dots \rightarrow \tau_n$$

where each step either corresponds to the execution of one instruction or traversing a control-flow edge from the post-point of one instruction to the pre-point of a successor instruction. Let us write “ $\text{zombie}(p, \tau_i)$ ” as shorthand for “ $\text{zombie}(p, t)$ ” where t is the program point to which τ_i corresponds. The inductive invariant has a few parts:

- **I_{SW} (Single writer):** Every memory location is pointed to by at most one write-enabled live C-path.
- **I_{UO} (Unique owner):** For every live heap block B , at most one live location holds a non-zombie responsible pointer to that block.
- **I_Z (Zombie state predicates are sound):** For every C-path p that is live at τ_i : If p concretely holds an uninitialized or freed pointer at τ_i , then $\text{zombie}(p, \tau_i)$ is true. (Note: the good predicate is needed to detect memory leaks, and the null predicate is needed to detect null pointer dereferences, but neither is needed to detect use-after-free or double-free.)
- **I_{BC} (Borrow info is complete):** For every live heap block B with a unique responsible owner o :
 - For every live C-path q (other than o itself) that concretely points to B , the current borrow information records that q is a transitive borrower of o .

Lemma: The above invariants hold true at each point in the execution trace.

Base case. In the base case, at the start of the program, all the invariants are trivially true.

Inductive case. We proceed by a case split on the instruction type. Most of the cases are straightforward, and we omit them for brevity. A few interesting cases we discuss below:

- “ $t = \text{load } p$ ” (where t is of pointer type):
 - C-paths with changed values are t , $*t$, $**t$, etc.
 - I_{SW} : To show that I_{SW} is preserved, we need to show that, for each $*^i t$, if it is live and write-enabled, then there isn’t another write-enabled live C-path pointing to the same memory location. By the inductive hypothesis, right before the load, if $*^{i+1}p$ is live, write-enabled, and points to a memory location L , then it is the only live write-enabled C-path that points to L , and after the load, it will no longer be write-enabled because it will have a borrow to $*^i t$, which in turn becomes write-enabled. If $*^{i+1}p$ isn’t live or write-enabled before the load, then neither will $*^i t$ after the load.
 - I_{UO} : The load instruction changes the value of a single location, namely that of the variable t . If t is irresponsible, then I_{UO} trivially still holds after the load. If t is responsible, then POM constraints zombieify the pointer stored in $*p$, so that t becomes the sole live location holding a non-zombie responsible pointer to the heap block that t points to.
 - I_Z : For C-paths rooted at something other than t , I_Z clearly still holds after the load. For C-paths rooted at t , the state propagation and deep-copy constraints ensure that I_Z still holds after the load if it held before the load.
 - I_{BC} : If t is responsible, then POM constraints permit the load only if there are no outstanding borrows from $*p$, so there is no borrow information to update. If t is irresponsible, then POM records a borrow from $*p$ to t , from $**p$ to $*t$, and so on up thru the pointer depth from the type information, ensuring that I_{BC} remains valid after the load.
- “store t , p ”:
 - The store writes the pointer value in t into the memory location pointed to by p . Thus the C-paths whose values may change are $*p$, $**p$, $***p$, etc., together with their location aliases.
 - I_{SW} : The instruction writes the value of t to the location pointed to by p . POM constraints require p to be write-enabled. Therefore, by the inductive hypothesis for I_{SW} , no other write-enabled C-path points to the same memory location as p . When t is copied to $*p$, if t is responsible, it is zombieified (move semantics), leaving $*p$ as the sole write-enabled path.

If t is irresponsible, a borrow is recorded from t to $*p$ (and recursively for deeper dereferences), which removes write capabilities from t while granting them to $*p$ (and recursively for deeper dereferences).

- I_{UO} : If t and $*p$ are responsible, the constraints enforce move semantics, meaning t gets marked as a zombie, and $*p$ becomes the new unique owner.
- I_Z : State propagation constraints require $*^{i+1}p$ (and its location aliases) to inherit the zombieness of $*^i t$ for $i \in \{0, \dots, \text{ptr_depth}\}$.
- I_{BC} : If t is responsible, POM requires it to have no outstanding borrows prior to the move, so there is no borrow information to update. If t is irresponsible, POM records a direct borrow from $*^i t$ to $*^{i+1}p$ for $i \in \{0, \dots, \text{ptr_depth}\}$.
- “free(p)”:
 - The concrete effect of free(p) is to mark the heap block pointed to by p as freed. Also, p gets marked as a zombie (unless p is NULL, in which case the free instruction is a no-op).
 - I_{SW} : We need to establish that any newly write-enabled live C-paths do not point to a memory location that is also pointed to by another write-enabled live C-path. Suppose that, before the free, there was a live write-enabled C-path r pointing to a location L , and this C-path r gets killed by the free. By the inductive hypothesis, before the free, r was the only write-enabled live C-path that pointed to L . After the free, the C-path that r was borrowing from (let’s call it q) now has no outstanding borrows, so q becomes write-enabled, and the number of live write-enabled C-paths pointing to L remains 1.
 - I_{UO} : The free instruction does not change any pointer states from zombie to non-zombie, nor does it introduce new heap blocks or change the contents of existing heap blocks, so I_{UO} is preserved.
 - I_Z : If the concrete value stored in p is NULL, then the instruction is a no-op, and I_Z is trivially preserved. Otherwise, if p is not NULL, then the concrete value stored in p after the free is a dangling pointer to the freed block, and p gets marked as a zombie. By I_{UO} , p was the unique responsible owner of B . There cannot be any other live C-path pointing to the freed block, because POM constraints forbid calling free on a C-path with outstanding borrows. Therefore every live C-path that concretely holds a dangling pointer after the free is marked zombie, so I_Z is preserved.
 - I_{BC} : The borrow information is updated as described above for I_{SW} .

References

- [1] [n. d.]. LLVM. <https://llvm.org> [Version 15.0.7 on Ubuntu 24.04].
- [2] [n. d.]. Rust Compiler Development Guide, IR borrow check. <https://rustc-dev-guide.rust-lang.org/borrow-check.html>
- [3] Robert C. Armstrong, Noah Evans, Geoffrey Compton Hulette, Karla Vanessa Morris, Jon M. Aytac, Philip Alden Johnson-Freyd, Jackson Mayo, and Ratish J. Punnoose. 2019. *Dishwashers of Armageddon: Verifying High Consequence Systems for Nuclear Weapons*. Technical Report. Sandia National Laboratories.
- [4] Roberto Bagnara, Abramo Bagnara, and Federico Serafini. 2023. *C-rusted: The advantages of Rust, in C, without the disadvantages*. arXiv:2302.05331 doi:10.48550/arXiv.2302.05331
- [5] Aron Baráth and Zoltán Porkoláb. 2015. Automatic checking of the usage of the C++ move semantics. *Acta Cybernetica* 22, 1 (2015), 5–20.
- [6] Paul E. Black. 2018. *Juliet 1.3 Test Suite*. Technical Report. National Institute of Standards and Technology.
- [7] Kirsten Bradley and Michael Godfrey. 2019. A study on the effects of exception usage in open-source C++ systems. In *2019 19th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 1–11.
- [8] Marek Chalupa, Jan Strejček, and Martina Vitovská. 2018. Joint Forces for Memory Safety Checking. In *International Symposium on Model Checking Software*. Springer, 115–132.
- [9] Marek Chalupa, Jan Strejček, and Martina Vitovská. 2020. Joint forces for memory safety checking revisited. *International Journal on Software Tools for Technology Transfer* 22, 2 (2020), 115–133.
- [10] Zhe Chen, Chuanqi Tao, Zhiyi Zhang, and Zhibin Yang. 2018. Beyond spatial and temporal memory safety. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*. ACM, 189–190. doi:10.1145/3183440.3195090
- [11] Zhe Chen, Jun Wu, Qi Zhang, and Jingling Xue. 2022. A dynamic analysis tool for memory safety based on smart status and source-level instrumentation. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 6–10.
- [12] Zhe Chen, Qi Zhang, Jun Wu, Junqi Yan, and Jingling Xue. 2022. A source-level instrumentation framework for the dynamic analysis of memory safety. *IEEE Transactions on Software Engineering* 49, 4 (2022), 2107–2127.
- [13] Luis Diogo Couto, Peter W.V. Tran-Jørgensen, René S. Nilsson, and Peter Gorm Larsen. 2020. Enabling continuous integration in a formal methods setting. *International Journal on Software Tools for Technology Transfer* 22 (2020), 667–683.
- [14] cppreference.com. [n. d.]. RAI. <https://en.cppreference.com/w/cpp/language/rai>. [Online; accessed 15-May-2025].
- [15] DARPA. [n. d.]. TRACTOR: translating all C to Rust. <https://www.darpa.mil/research/programs/translating-all-c-to-rust>. Accessed May 29, 2025.
- [16] Archibald Samuel Elliott, Andrew Ruef, Michael Hicks, and David Tarditi. 2018. Checked C: making C safe by extension. In *2018 IEEE Cybersecurity Development (SecDev)*. IEEE Computer Society, 53–60. doi:10.1109/SECDEV.2018.00015
- [17] Nathaniel Wesley Filardo, Brett F. Gutstein, Jonathan Woodruff, Sam Ainsworth, Lucian Paul-Trifu, Brooks Davis, Hongyan Xia, Edward Tomasz Napierala, Alexander Richardson, John Baldwin, et al. 2020. Cornucopia: temporal safety for CHERI heaps. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 608–625.
- [18] Nathaniel Wesley Filardo, Brett F. Gutstein, Jonathan Woodruff, Jessica Clarke, Peter Rugg, Brooks Davis, Mark Johnston, Robert Norton, David Chisnall, Simon W. Moore, et al. 2024. Cornucopia reloaded: load barriers for CHERI heap temporal safety. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 251–268.
- [19] Immunant. [n. d.]. C2Rust: Migrate C code to Rust. <https://github.com/immunant/c2rust>. GitHub repository. Accessed: 2026-05-12.
- [20] International Organization for Standardization. 2018. *ISO/IEC 9899:2018 Information technology — Programming languages — C*. Technical Report ISO/IEC 9899:2018. ISO.
- [21] Trevor Jim, J Gregory Morrisett, Dan Grossman, Michael W Hicks, James Cheney, and Yanling Wang. 2002. Cyclone: a safe dialect of C.. In *USENIX ATC, General Track*. 275–288.
- [22] Kerry Doyle and Twain Taylor. [n. d.]. Rust vs. C++: differences and use cases explained. <https://www.techtarget.com/searchapparchitecture/tip/Rust-vs-C-Differences-and-use-cases>. [Online; accessed May 15, 2025].
- [23] Taddeus Kroes, Koen Koning, Erik van der Kouwe, Herbert Bos, and Cristiano Giuffrida. 2018. Delta pointers: buffer overflow checks without the checks. In *Proceedings of the Thirteenth EuroSys Conference, EuroSys 2018*, Rui Oliveira, Pascal Felber, and Y. Charlie Hu (Eds.). ACM, 22:1–22:14. doi:10.1145/3190508.3190553
- [24] D. Richard Kuhn, Ramaswamy Chandramouli, and Ricky W. Butler. 2002. Cost-effective use of formal methods in verification and validation foundations. In *2002 V&V Workshop, National Institute of Standards and Technology*.
- [25] Rodolphe Lepigre, Michael Sammler, Kayvan Memarian, Robbert Krebbers, Derek Dreyer, and Peter Sewell. 2022. VIP: verifying real-world C idioms with integer-pointer casts. *Proceedings of the ACM on Programming Languages* 6, POPL (2022), 1–32.
- [26] Aravind Machiry, John Kastner, Matt McCutchen, Aaron Eline, Kyle Headley, and Michael Hicks. 2022. C to Checked C by 3C. *Proceedings of the ACM on Programming Languages* 6, OOPSLA1 (2022), 1–29.
- [27] David Manners. 2024. CHERI Alliance launched. <https://www.electronicshweekly.com/news/business/cheri-alliance-launched-2024-11/>. Accessed: Jan. 20, 2025.
- [28] Jackson R. Mayo, Karla Vanessa Morris Wright, Jon M. Aytac, Andrew Michael Smith, Robert Claude Armstrong, Geoffrey Compton Hulette, and Randall R. Lober. 2024. *Demonstration of Model-Based Design for Digital Controller Using Formal Methods*. Technical Report. Sandia National Laboratories.
- [29] MITRE. 2006. CWE-401: Missing Release of Memory after Effective Lifetime. <https://cwe.mitre.org/data/definitions/401.html>. Accessed May 29, 2025.
- [30] MITRE. 2006. CWE-415: Double Free. <https://cwe.mitre.org/data/definitions/415.html>. Accessed May 29, 2025.
- [31] MITRE. 2006. CWE-416: Use After Free. <https://cwe.mitre.org/data/definitions/416.html>. Accessed May 29, 2025.
- [32] MITRE. 2006. CWE-590: Free of Memory not on the Heap. <https://cwe.mitre.org/data/definitions/590.html>. Accessed May 29, 2025.
- [33] MITRE. 2009. CWE-761: Free of Pointer not at Start of Buffer. <https://cwe.mitre.org/data/definitions/761.html>. Accessed May 29, 2025.
- [34] MITRE. 2025. 2025 CWE Top 25 Most Dangerous Software Weaknesses. https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html. Accessed 2026-03-16.
- [35] Nausheen Mohammed, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. 2024. *Enabling memory safety of C programs using LLMs*. arXiv:2404.01096 doi:10.48550/arXiv.2404.01096
- [36] Karla Vanessa Morris, Colin Snook, Thai Son Hoang, Robert C. Armstrong, Geoffrey Compton Hulette, and Michael Butler. 2020. *Refinement of Reactive Systems*. Technical Report. Sandia National Laboratories.
- [37] Karla Vanessa Morris Wright, Thai Son Hoang, Colin Snook, and Michael Butler. 2023. Formal language semantics for triggered enable statecharts with a run-to-completion scheduling. In *International Colloquium on Theoretical Aspects of Computing*. Springer, 178–195.
- [38] Santosh Nagarakatte. 2024. Full spatial and temporal memory safety for C. *IEEE Security & Privacy* 22, 4 (2024), 30–39.
- [39] László Nagy. [n. d.]. Bear. <https://github.com/rizotto/Bear> [Version 3.1.3 on Ubuntu 24.04].
- [40] George C Necula, Jeremy Condit, Matthew Harren, Scott McPeak, and Westley Weimer. 2005. CCured: Type-safe retrofitting of legacy

- software. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 27, 3 (2005), 477–526.
- [41] Thomas Neele and Anton Wijs. 2024. *Model Checking Software*. Springer.
- [42] OpenAI. 2025. o4-mini. <https://developers.openai.com/api/docs/models/o4-mini>. OpenAI API model documentation. Accessed 2026-03-18.
- [43] Samuel D. Pollard, Robert C. Armstrong, John Bender, Geoffrey C. Hulette, Raheel S. Mahmood, Karla Morris, Blake C. Rawlings, and Jon M. Aytac. 2022. Q: a sound verification framework for statecharts and their implementations. In *Proceedings of the 8th ACM SIGPLAN International Workshop on Formal Techniques for Safety-Critical Systems*. 16–26.
- [44] Christopher Pulte, Dhruv C. Makwana, Thomas Sewell, Kayvan Memarian, Peter Sewell, and Neel Krishnaswami. 2023. CN: verifying systems C code with separation-logic refinement types. *Proceedings of the ACM on Programming Languages* 7, POPL (2023), 1–32.
- [45] G. Ramalingam. 1994. The undecidability of aliasing. *ACM Transactions on Programming Languages and Systems* 16, 5 (1994), 1467–1471.
- [46] Tiago Silva, João Bispo, and Tiago Carvalho. 2024. Foundations for a Rust-like borrow checker for C. In *Proceedings of the 25th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*. 155–165.
- [47] Matthew S. Simpson and Rajeev K. Barua. 2013. MemSafe: ensuring the spatial and temporal memory safety of C at runtime. *Software: Practice and Experience* 43, 1 (2013), 93–128.
- [48] Software Engineering Institute. 2026. SEI CERT Coding Standards. <https://cmu-sei.github.io/secure-coding-standards>. Accessed May 15, 2026.
- [49] Mate Soos, Karsten Nohl, and Claude Castelluccia. 2009. Extending SAT Solvers to Cryptographic Problems. In *Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009, Swansea, UK, June 30 - July 3, 2009. Proceedings (Lecture Notes in Computer Science, Vol. 5584)*, Oliver Kullmann (Ed.). Springer, 244–257. doi:10.1007/978-3-642-02777-2_24
- [50] Bjarne Stroustrup. 2013. *The C++ Programming Language* (4 ed.). Addison-Wesley.
- [51] David Svoboda, Will Klieber, Jeffrey Hoskinson, Lori Flynn, and Ruben Martins. 2026. Reproduction Artifact for a Pointer-Ownership Model for C Inspired by Rust. Zenodo. doi:10.5281/zenodo.20158172
- [52] David Svoboda, Will Klieber, Jeffrey Hoskinson, Lori Flynn, and Sasank Vishnubhatla. 2025. Pointer Ownership Model (POM) Project. <https://github.com/cmu-sei/pom>. GitHub repository. Accessed: 2026-05-12.
- [53] David Svoboda and Lutz Wrage. 2014. Pointer ownership model. In *2014 47th Hawaii International Conference on System Sciences*. IEEE, 5090–5099.
- [54] Laszlo Szekeres, Mathias Payer, Tao Wei, and Dawn Song. 2013. SoK: eternal war in memory. In *2013 IEEE Symposium on Security and Privacy*. IEEE, 48–62.
- [55] Maurice H. ter Beek, Rod Chapman, Rance Cleaveland, Hubert Garavel, Rong Gu, Ivo ter Horst, Jeroen J.A. Keiren, Thierry Lecomte, Michael Leuschel, Kristin Yvonne Rozier, et al. 2024. Formal methods in industry. *Formal Aspects of Computing* 37, 1 (2024), 1–38.
- [56] Robert N.M. Watson, Peter G. Neumann, Jonathan Woodruff, Michael Roe, Hesham Almatary, Jonathan Anderson, John Baldwin, David Chisnall, Brooks Davis, Nathaniel Wesley Filardo, et al. 2019. *Capability Hardware Enhanced RISC Instructions: ChERI instruction-set architecture (version 7)*. Technical Report. University of Cambridge, Computer Laboratory.
- [57] Nathan Wetzler, Marijn JH Heule, and Warren A Hunt Jr. 2014. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *International Conference on Theory and Applications of Satisfiability Testing*. Springer, 422–429.
- [58] Hongyan Xia, Jonathan Woodruff, Sam Ainsworth, Nathaniel W. Filardo, Michael Roe, Alexander Richardson, Peter Rugg, Peter G. Neumann, Simon W. Moore, Robert N.M. Watson, et al. 2019. ChERiVoke: characterising pointer revocation using ChERI capabilities for temporal memory safety. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 545–557.
- [59] Xiaohua Yin, Zhiqiu Huang, Shuanglong Kan, and Guohua Shen. 2024. Safemd: Ownership-based safe memory deallocation for c programs. *Electronics* 13, 21 (2024), 4307.
- [60] Vadim Zaliva, Kayvan Memarian, Brian Campbell, Ricardo Almeida, Nathaniel Filardo, Ian Stark, and Peter Sewell. 2025. A ChERI C memory model for verified temporal safety. In *Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, 112–126.
- [61] Anna Zamansky, Maria Spichkova, Guillermo Rodriguez-Navas, Peter Herrmann, and Jan Olaf Blech. 2018. *Towards classification of lightweight formal methods*. arXiv:1807.01923 doi:10.48550/arXiv.1807.01923
- [62] Hanliang Zhang, Cristina David, Yijun Yu, and Meng Wang. 2023. Ownership guided C to Rust translation. In *International Conference on Computer Aided Verification*. Springer, 459–482.
- [63] Tong Zhang, Dongyoon Lee, and Changhee Jung. 2019. BOGO: buy spatial memory safety, get temporal memory safety (almost) free. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 631–644.
- [64] Jie Zhou, John Criswell, and Michael Hicks. 2023. Fat pointers for temporal memory safety of C. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 316–347.

Received 2026-03-05; accepted 2026-05-01